# Chapter 4
# File Systems

- Files
- Directories
- File System Implementation
- Example File Systems

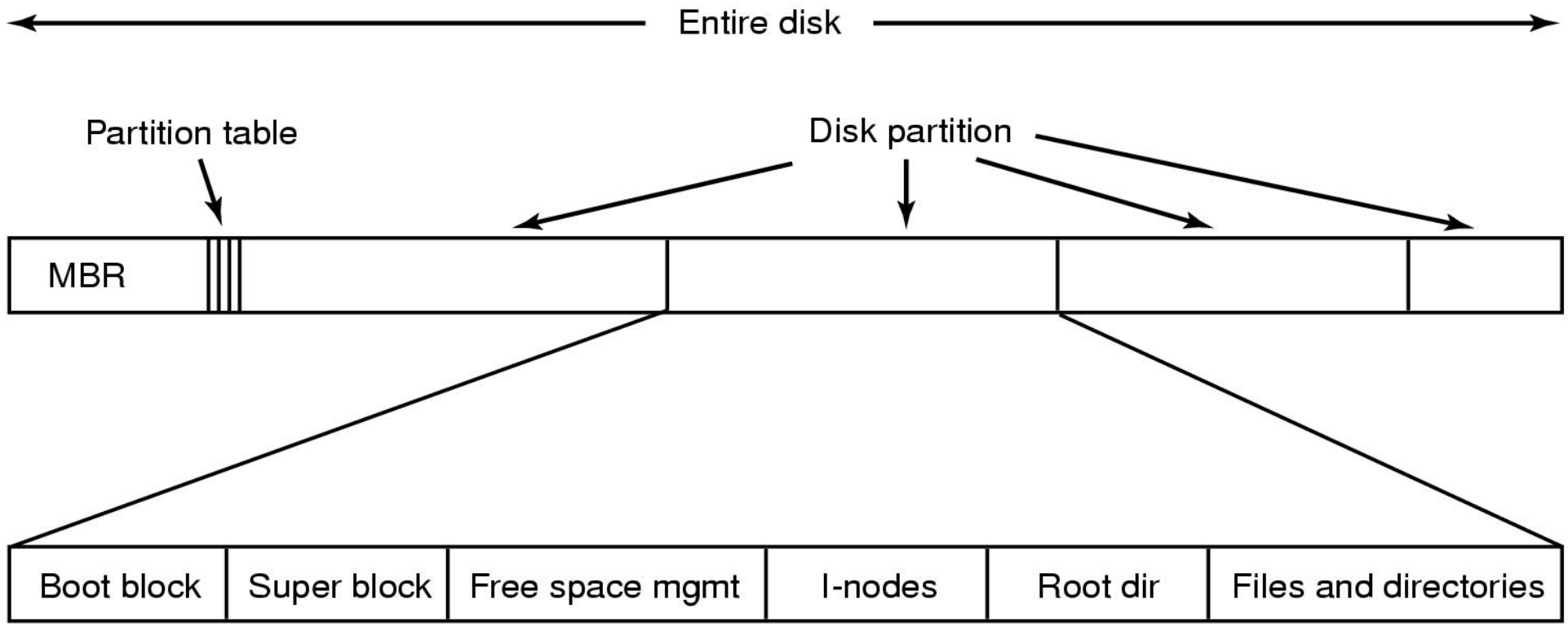
# File System Implementation

- Contents
  - File System Layout
  - Implementing Files
  - Implementing Directory
  - Shared Files
  - Disk Space Management
  - File System Reliability
  - File System Performance

# File System Layout (1)

- MBR (Master Boot Record) in sector 0 is used to boot the computer.
- The **partition table** gives the starting and ending addresses of each partition, one of which is marked as active
- Each partition can hold its own file system, Unix file system, Window file system or others
- Every partition starts with a “boot block”
  - The boot block contains a small program, which can load the OS in that partition
- OS Startup
  - BIOS reads MBR, then locates the active partition, reads in its boot block and executes it

# File System layout (2)



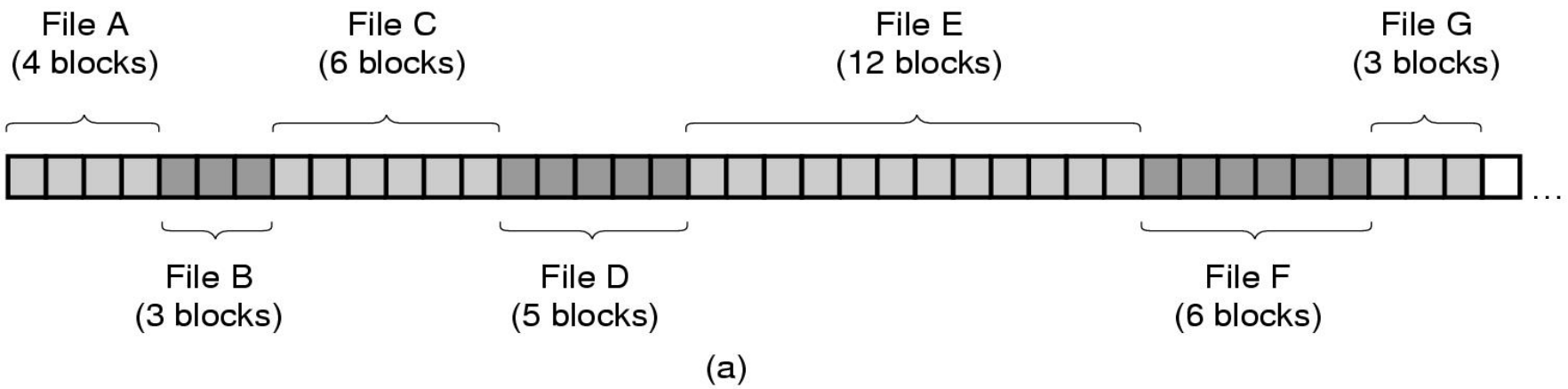

A possible file system layout

# Implementing files

- Implementing file storage is keeping track of which disk blocks go with which files.
- Three methods
  - Contiguous Allocation
  - Linked List Allocation
  - Indexed Allocation

# Contiguous Allocation (1)

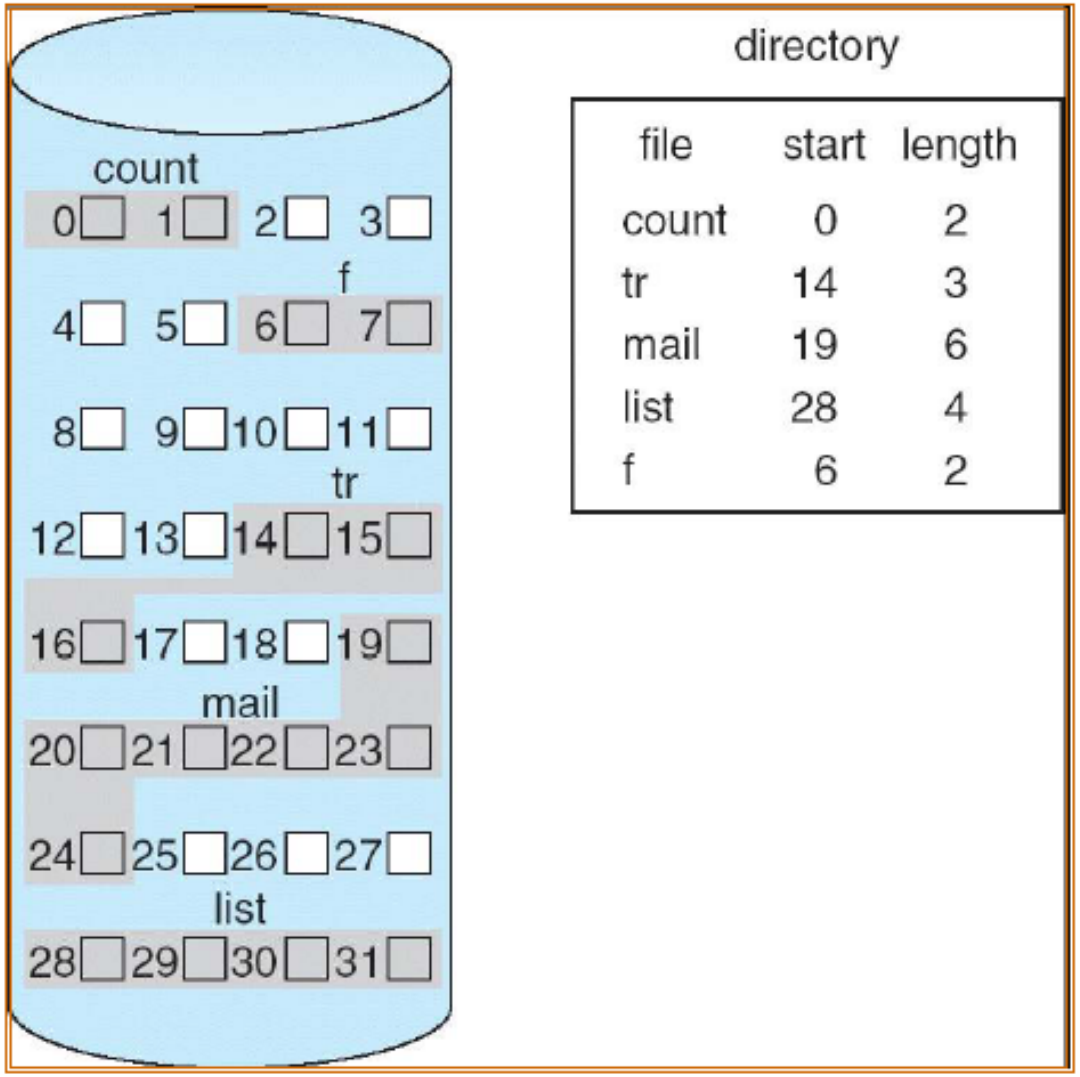
- Store each file as contiguous block of data.




(a) Contiguous allocation of disk space for 7 files

(b) State of the disk after files *D* and *F* have been removed

# Contiguous Allocation (2)

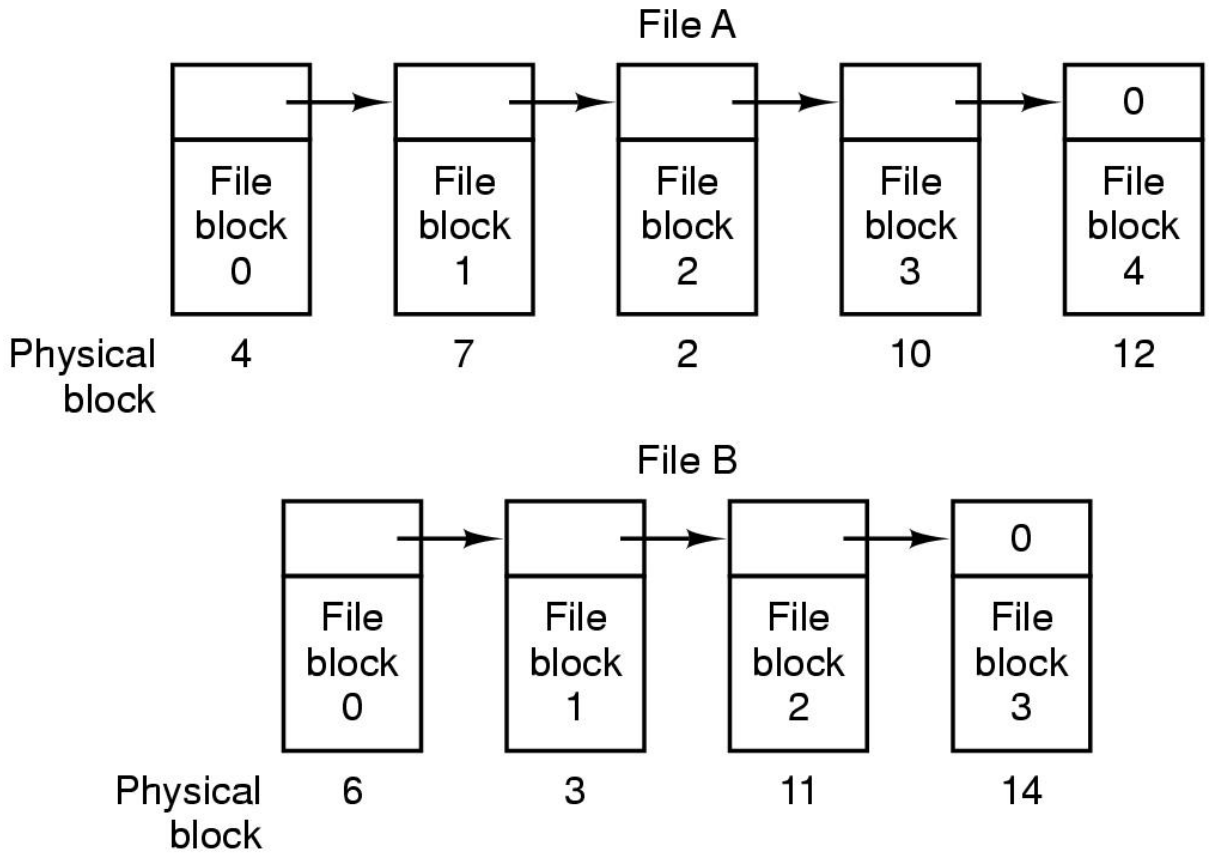


# Contiguous Allocation (3)

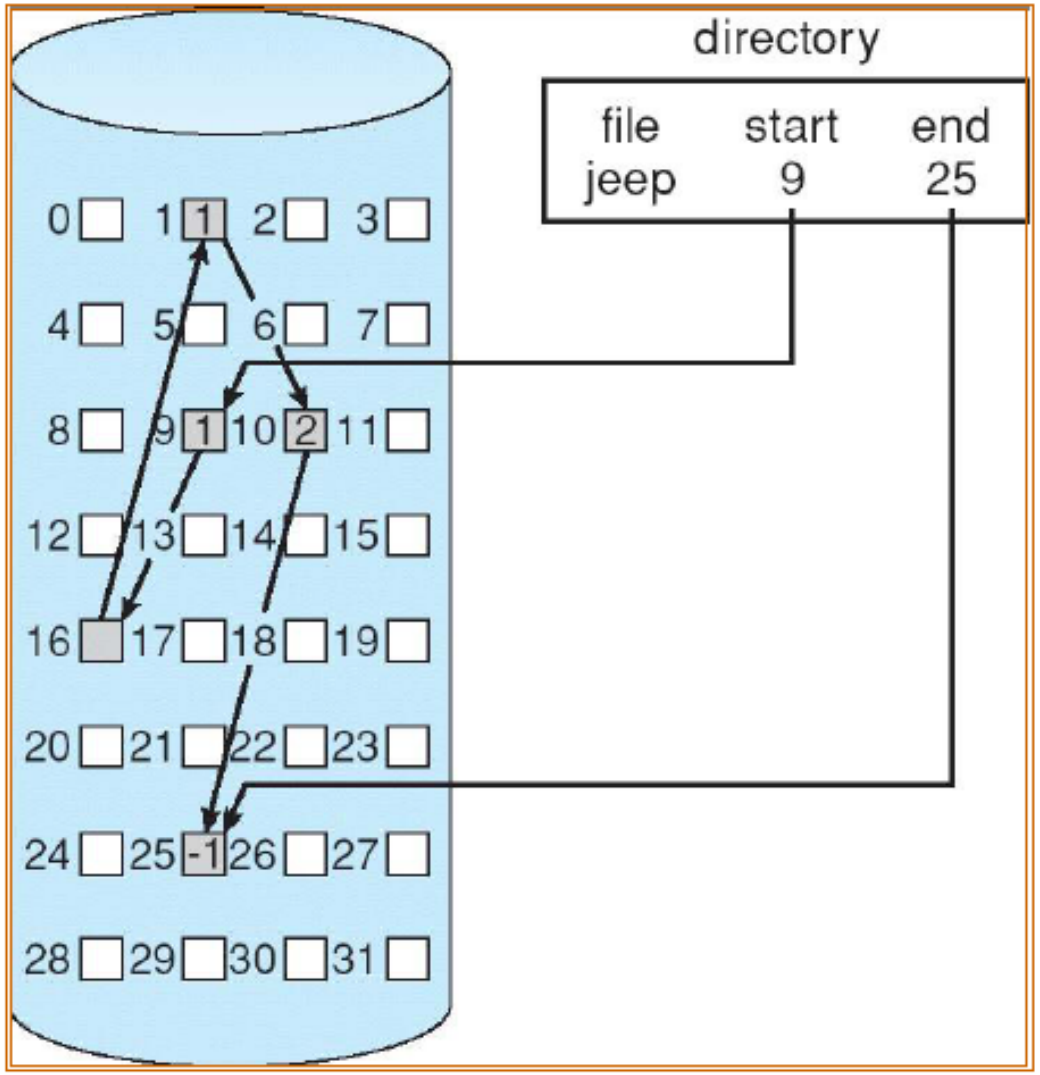
- Advantages
  - Simple to implement (Need only starting sector & length of file)
  - Read performance is excellent (suits for sequential access and direct access)
- Disadvantages
  - Disk fragmentation
  - The maximum file size must be known when file is created, or files cannot grow
- Good for CD-ROMs, DVDs and other write-once optical media
  - All file sizes are known in advance
  - Files are never deleted

# Linked List Allocation (1)

- Storing a file as a linked list of disk blocks, blocks may be scattered anywhere on the disk.
- The first word of each block is used as a pointer to the next one.



# Linked List Allocation (2)



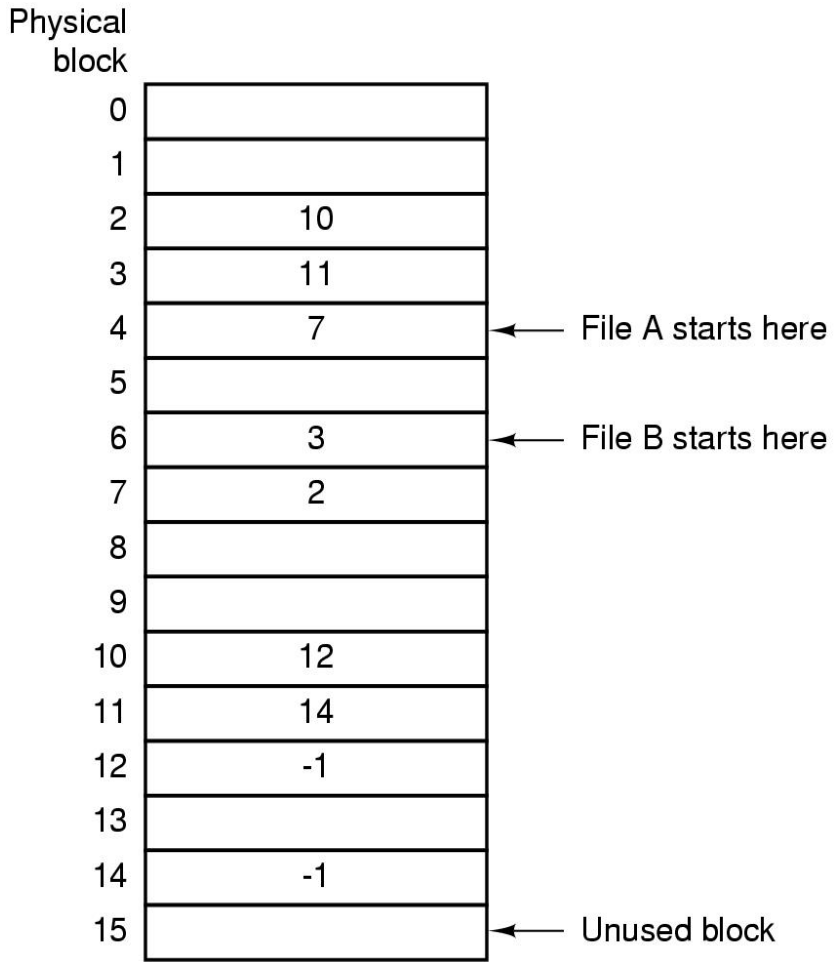
# Linked List Allocation (3)

- Advantages:
  - No external fragmentation
  - Directory entries are simple (Need only starting address)
  - A file can continue to grow as long as free blocks are available
  - Good for sequential access
- Disadvantages:
  - Random access slow
  - The amount of data in a block is not a power of 2

# Linked List Allocation Using FAT (1)

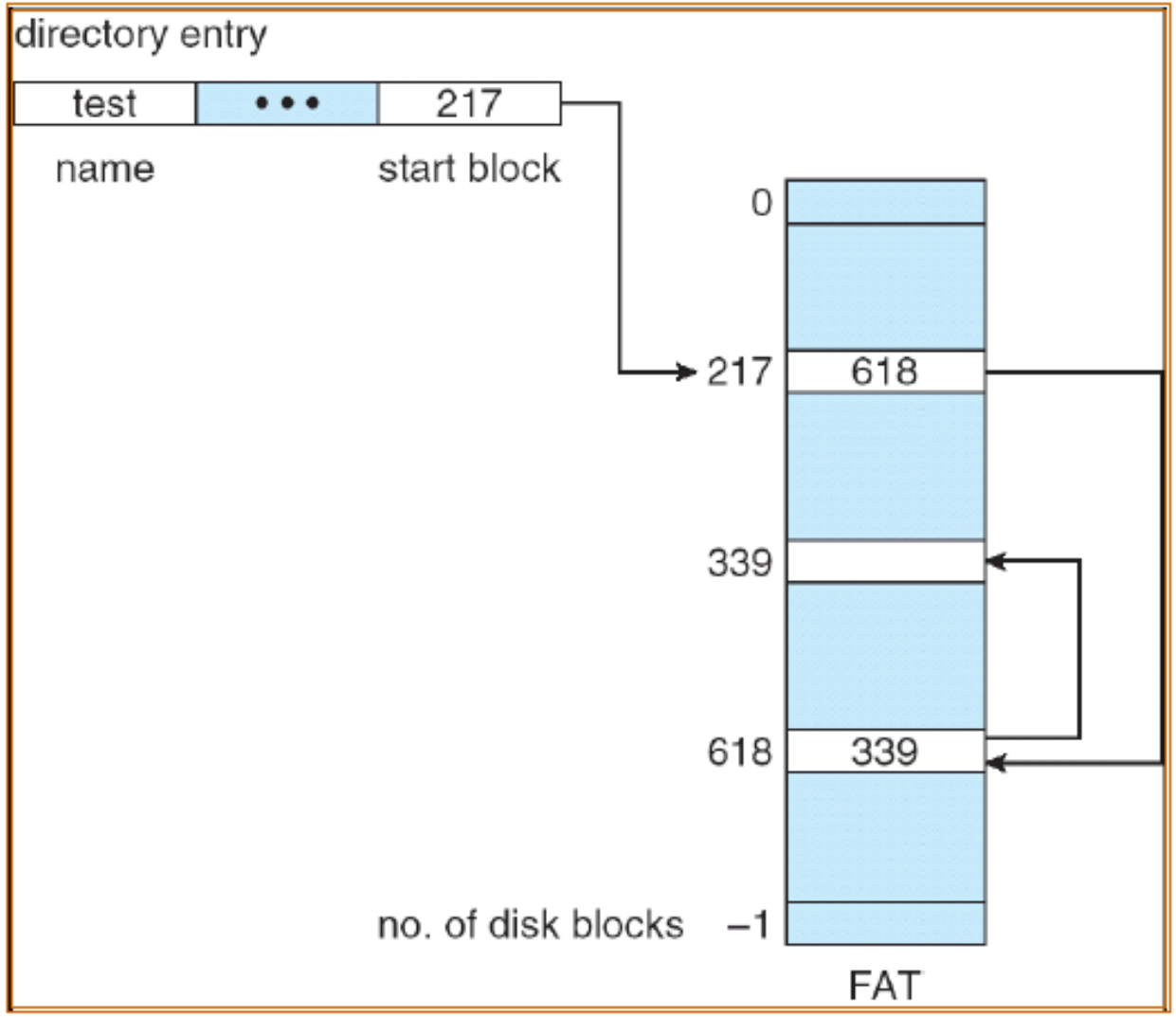
- Take *table pointer word* from each block and put them in an index table, FAT (File Allocation Table).
- A section of disk at the beginning of each partition is set aside to contain FAT
- FAT should be kept in memory to minimize disk seeks
  - Space overhead can be substantial
- File Allocation Table
  - One entry per block on the disk, ordered by block number
  - Each entry contains the address of the “next” block
  - End of file marker (-1)
  - A special value (e.g. -2) indicates the block is free
- Used by OS/2 and MS-DOS

# Linked List Allocation Using FAT (2)




Linked list allocation using FAT in RAM

# Linked List Allocation Using FAT (3)



# Linked List Allocation Using FAT (4)

- Advantages
  - The entire block is available for data
  - Random access...
    - Search the linked list (but all in memory)
  - Directory Entry needs only one number
    - Starting block number
- Disadvantage
  - Entire table must be in memory all at once!
    - Example

      200 GB = disk size

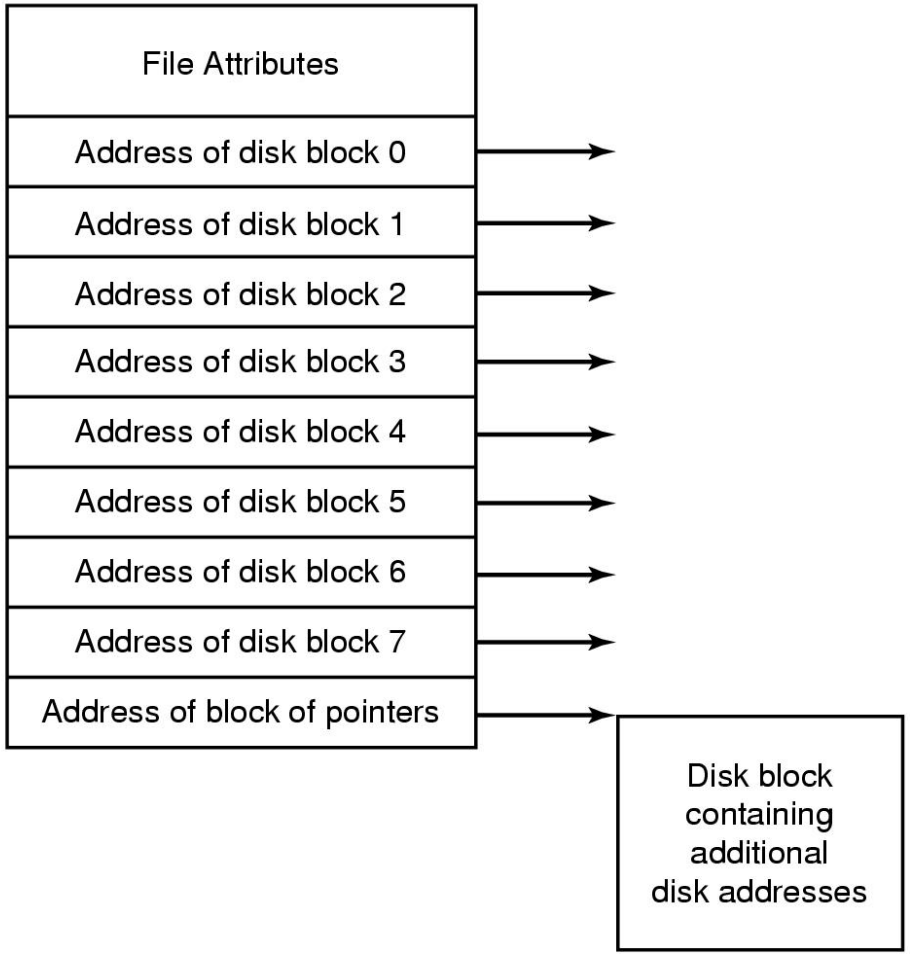
      1 KB = block size

      4 bytes = FAT entry size

      800 MB of memory used to store the FAT

# Indexed Allocation (i-Node) (1)

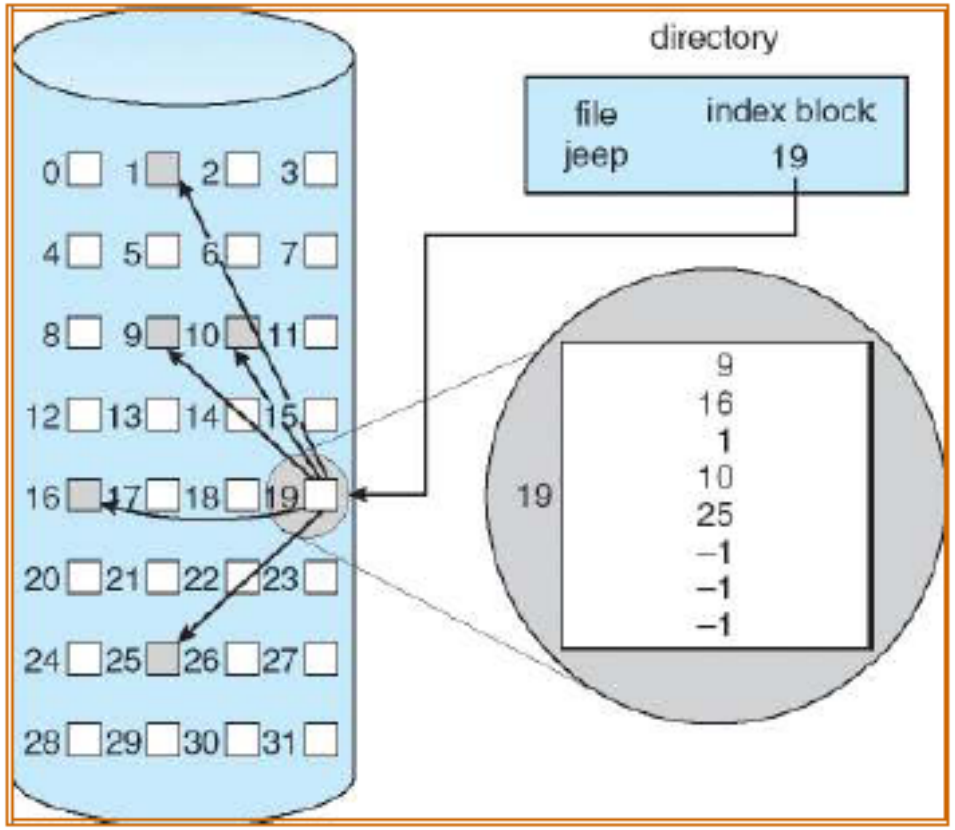
- Bring all the pointers together into one location (index block or index-node)
- Each file has its own index-node (i-node), which lists the attributes and disk block addresses of the file.




An example i-node

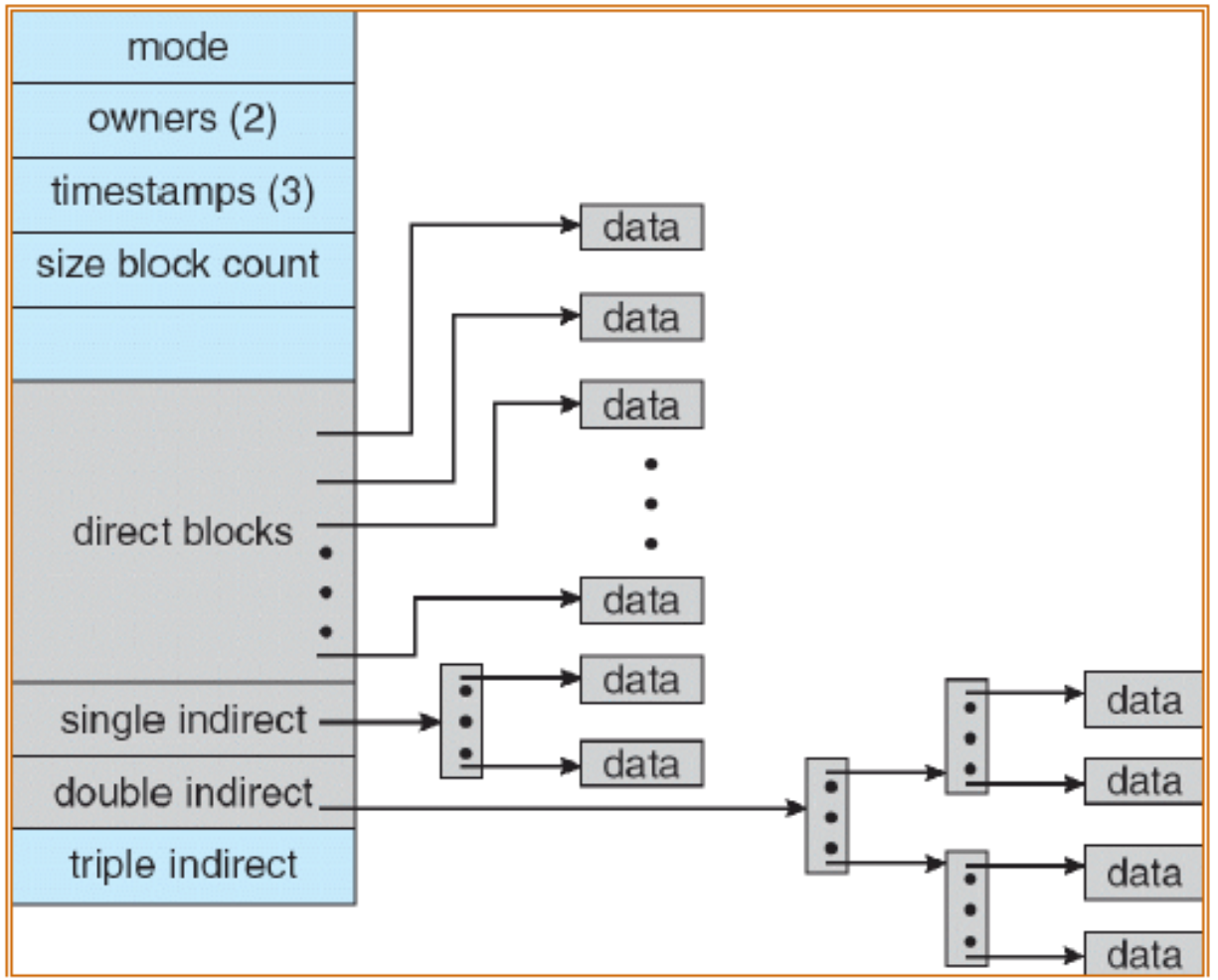
# Indexed Allocation (i-Node) (2)

- Loading the i-node need only into memory when the corresponding file is opened.
- Each directory entry contains i-number only
- Example:



# Indexed Allocation (i-Node) (3)

- Multi-level Indexed Allocation



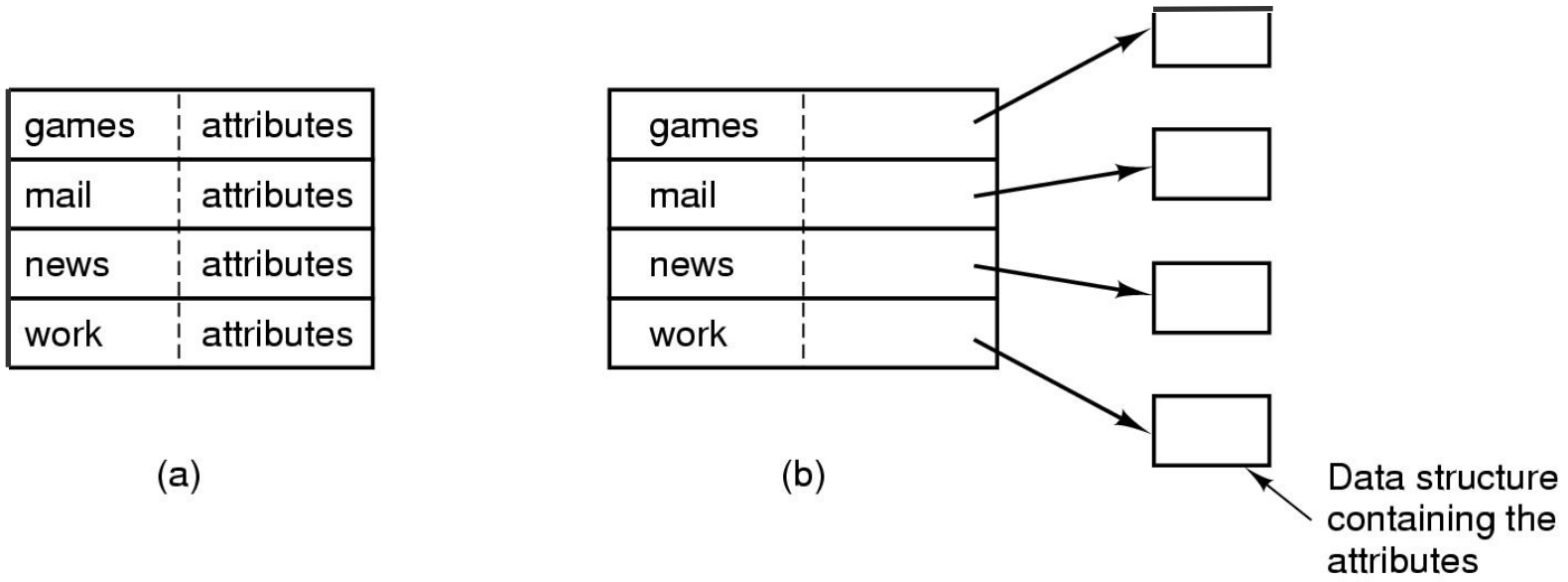
# Indexed Allocation (i-Node) (4)

- Advantages
  - Supports direct access
  - Without suffering from external fragmentation
  - i-node need only be in memory when the corresponding file is open
- Disadvantage
  - Space overhead for indexes

# Implementation Directories (1)

- When a file is opened, the file system uses the path name to locate the directory entry.
- The directory entry provides information needed to find the disk blocks.
  - disk address of the entire file (contiguous blocks)
  - the number of first block (linked list)
  - the number of i-node (i-node)
- Where to store attributes? In directory or i-node?

# Implementing Directories (2)

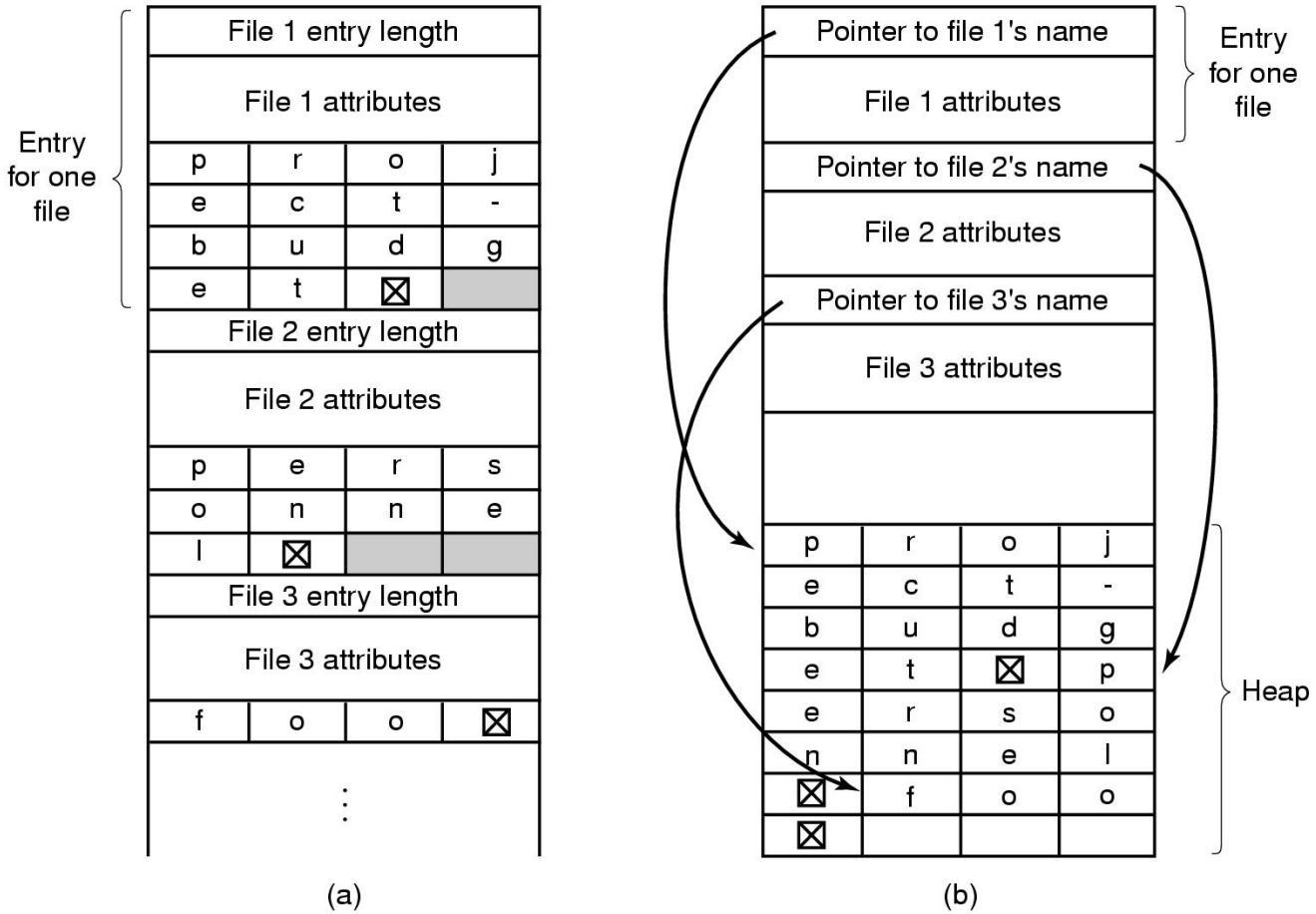



(a) A simple directory – MS-DOS/Windows
    fixed size entries
    disk addresses and attributes in directory entry
(b) Directory in which each entry just refers to an i-node - UNIX

# Implementation Directories (3)

- Handling long file names in a directory:
  - Fixed-length names (Waste space)
  - In-line (When a file is removed, a variable-sized gap is introduced.)
  - Heap (The heap management needs extra effort.)
- How to search files in each directory?
  - Linearly
  - Hash table
  - Cache the results of searches

# Implementing Directories (4)

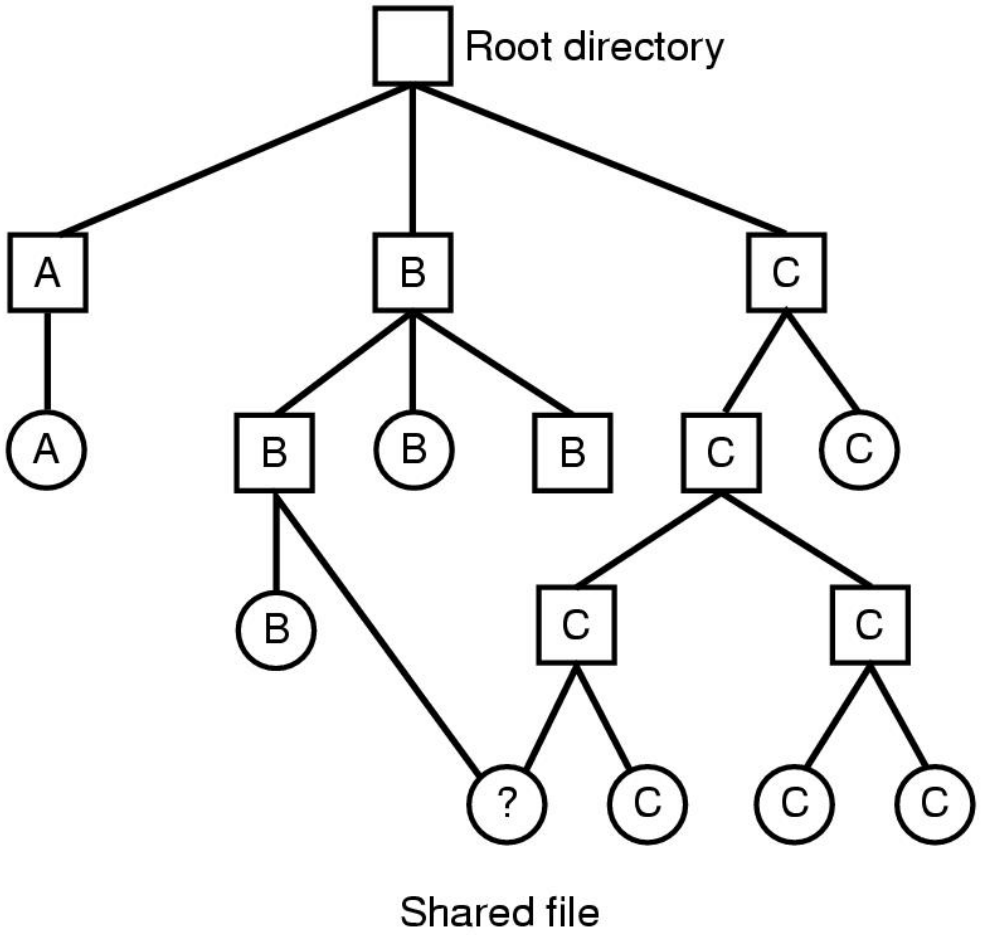



- Two ways of handling long file names in directory
  - (a) In-line
  - (b) In a heap

# Shared Files (1)

- A shared file is used to allow a file to appear in several directories.
- The connection between a directory and the shared file is called a **link**. The file system is a **Directed Acyclic Graph** (**DAG**), not a tree.
- Problems
  - If directories contain disk address, a copy of the disk address will have to be made in directory B.
  - If B or C append the file, the new blocks will only appear in one directory.

# Shared Files (2)




File system containing a shared file

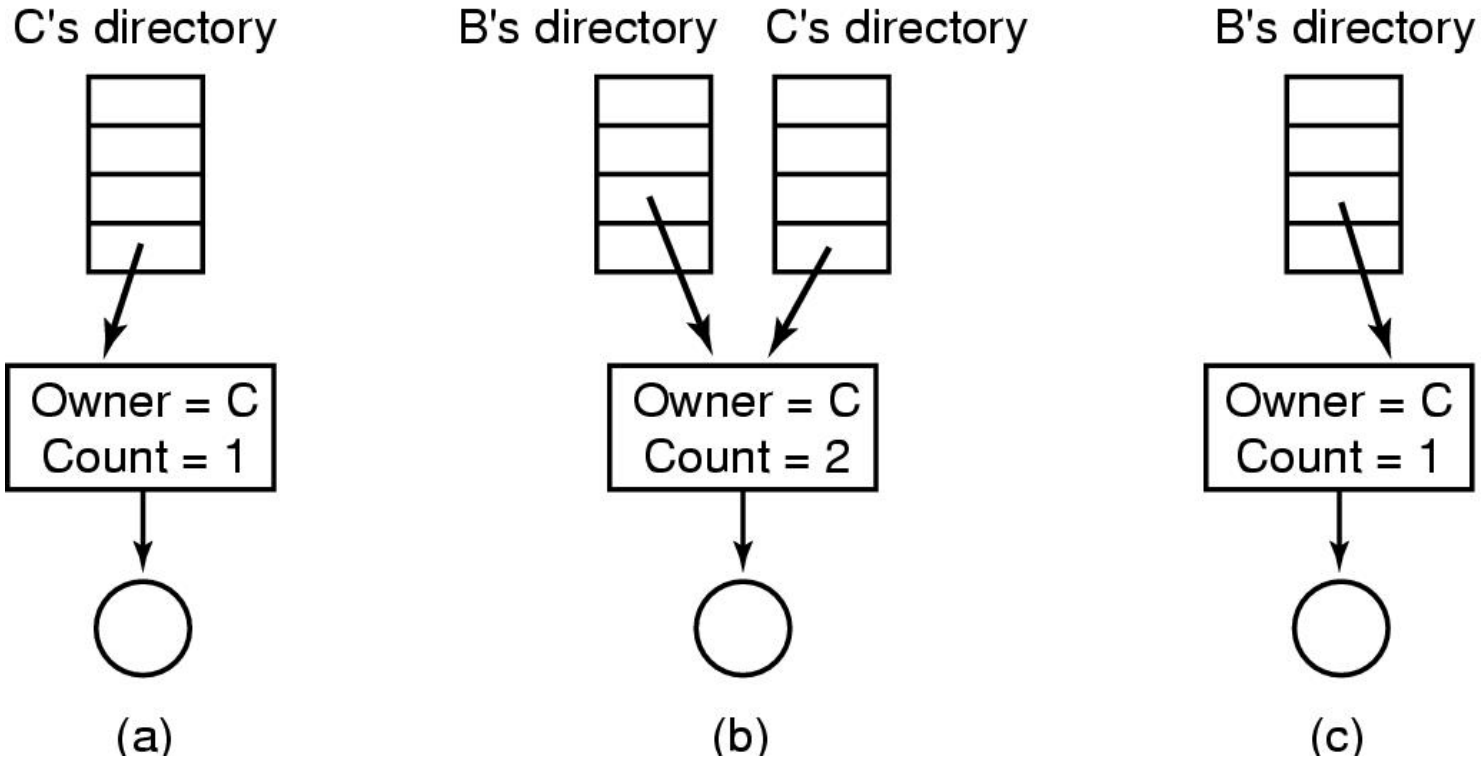
# Shared Files (3)

- Solution:
  - Do not list disk block addresses in directories but in a little data structure, i-node ➔ **Hard Link**
    - Both directories point to the same i-node (next page)
  - Create a new file of type LINK which contains the path name of the file to which it is linked ➔ **Symbolic Link** (Soft Link)
    - One directory points to the file's i-node
    - Other directory contains the "path"

# Shared Files (4)

- Deleting a file
  - Directory entry is removed from directory
  - All blocks in file are returned to free list
  - What about sharing???
    - Multiple links to one file (in Unix)
- Hard Links
  - Put a “reference count” field in each i-node
  - Counts number of directories that point to the file
  - When removing file from directory, decrement count
  - When count goes to zero, reclaim all blocks in the file
- Symbolic Link (only the owner directory has a pointer to i-node)
  - Remove the real file... (normal file deletion)
  - Symbolic link becomes “broken”

# Shared Files (5)




(a) Situation prior to linking

(b) After the link is created

(c) After the original owner removes the file

# Shared Files (6)

- Problems of symbolic link
  - Extra overhead in the traversing path
  - Having multiple copies of a file may set copied when dumping a file onto a tape.